RAPPORT DE CONCEPTION

Projet	StockManager CI
Contexte	Quincaillerie KOFFI & FRÈRES — Adjamé, Abidjan
Technologies	JavaFX 21 · JDBC · MySQL 8 · Maven · MVC + Repository
Date de remise	13 avril 2026
Groupe	**KOUAKOU Brou Fidèle - DOSSO Boiké – TRAORE Issiaka**

SOMMAIRE

RAPPORT DE CONCEPTION 1
1. Introduction 4
1.1 Contexte 4
1.2 Objectifs du projet 4
1.3 Périmètre fonctionnel 4
2. Architecture technique 5
2.1 Modèle MVC + Repository 5
Exemple de flux — Ajout d'un produit : 5
2.2 Structure des packages Maven 6
2.3 Choix techniques justifiés 7
2.4 Sécurité 7
3. Schéma de la base de données 8
3.1 Diagramme des classes 8
3.2 Description des tables 9
Table catégories 9
Table fournisseurs 9
Table produits 9
Table utilisateurs 10
Table mouvements 10
3.3 Contraintes d'intégrité 10
4. Description des fonctionnalités 11
4.1 Authentification 11
4.2 Module Catégories 12
4.3 Module Fournisseurs 13
4.4 Module Produits 13
4.5 Module Mouvements de Stock 14
4.6 Tableau de bord 15
4.7 Module Utilisateurs (ADMIN uniquement) 16
4.8 Pagination 17
5. Extensions implémentées 17
5.1 Splash Screen 17
5.2 Alertes visuelles stock minimum 17

5.3 Export XLSX (Apache POI) 17
5.4 Export PDF (bonus — iTextPDF) 17
5.5 Filtre par date et produit dans les mouvements 17
5.6 Tests unitaires JUnit 5 (bonus) 17
6. Tableau de répartition des tâches 18
7. Difficultés rencontrées et solutions 18
Difficulté 1 — `Location is not set` au démarrage 18
Difficulté 2 — ComboBox affichant l'adresse mémoire des objets 19
Difficulté 3 — Logo introuvable dans le Splash Screen 19
8. Bilan individuel 19
Dosso Boiké 19
Kouakou Fidèle 20
Traoré Issiaka 20
9. Conclusion et perspectives 21
9.1 Bilan 21
9.2 Perspectives d'amélioration 21
10. Annexes 22
Annexe A — Dépendances Maven 22
Annexe B — Identifiants de test 22
Annexe C — Commandes de déploiement 23
Annexe D — Extrait de code 23

1. Introduction

1.1 Contexte

Dans le tissu économique ivoirien et ouest-africain, la gestion manuelle des stocks représente l'une des principales sources de pertes pour les commerces, boutiques, pharmacies et petites entreprises. Les erreurs de comptage, les ruptures de stock non détectées et l'absence de traçabilité des entrées/sorties de marchandises sont des problèmes récurrents.

Ce projet nous a conduits à concevoir et développer **StockManager CI**, une application de bureau permettant à une structure commerciale ivoirienne de gérer ses stocks de façon informatisée, fiable et ergonomique. Le commerce retenu pour illustrer le contexte est la **Quincaillerie KOFFI & FRÈRES**, située à Adjamé, Abidjan.

1.2 Objectifs du projet

L'application vise à :

- Centraliser la gestion des produits, catégories et fournisseurs dans une base de données MySQL.
- Enregistrer tous les mouvements de stock (entrées et sorties) avec traçabilité complète.
- Alerter visuellement dès qu'un produit atteint son seuil de stock minimum.
- Fournir un tableau de bord synthétique avec les indicateurs clés de gestion.
- Sécuriser l'accès via un système d'authentification avec gestion des rôles.
- Exporter les données au format Excel (.xlsx) et PDF.

1.3 Périmètre fonctionnel

L'application couvre 9 modules :

a) authentification,
b) catégories,
c) fournisseurs,
d) produits,
e) mouvements de stock,
f) tableau de bord,
g) export,
h) utilisateurs
i) pagination.

Tous ces modules sont obligatoires selon le cahier des charges et ont été intégralement implémentés.

2. Architecture technique

2.1 Modèle MVC + Repository

L'application respecte strictement le modèle **MVC (Modèle-Vue-Contrôleur)** enrichi du modèle **Repository**. Ce dernier ajoute une couche d'abstraction entre les contrôleurs et la base de données, ce qui facilite la maintenabilité et les tests unitaires.

Vue
↕
Contrôleur
↕
Interface Repository
↕
DAO (implémentation SQL)
↕
MySQL 8

Exemple de flux — Ajout d'un produit :

a) L'utilisateur remplit le formulaire et clique sur « + Ajouter ».
b) `ProduitController.handleAjouter()` lit les champs et crée un objet `Produit`.
c) `produitDAO.save(produit)` est appelé via l'interface `IProduitRepository`.
d) `ProduitDAO` exécute le `INSERT` SQL avec un `PreparedStatement`.
e) MySQL enregistre la ligne dans la table `produits`.
f) Le contrôleur rafraîchit le `TableView` via `produitDAO.findAll(page, size)`

2.2 Structure des packages Maven

```
stockmanager-ci/
├── pom.xml
└── src/main/
    ├── java/com/inphb/icgl/stocks/
    │   ├── MainApp.java
    │   ├── SplashScreen.java
    │   ├── model/
    │   │   ├── Utilisateur.java
    │   │   ├── Categorie.java
    │   │   ├── Fournisseur.java
    │   │   ├── Produit.java
    │   │   └── Mouvement.java
    │   ├── repository/
    │   │   ├── ICategorieRepository.java
    │   │   ├── IFournisseurRepository.java
    │   │   ├── IProduitRepository.java
    │   │   ├── IMouvementRepository.java
    │   │   └── IUtilisateurRepository.java
    │   ├── dao/
    │   │   ├── CategorieDAO.java
    │   │   ├── FournisseurDAO.java
    │   │   ├── ProduitDAO.java
    │   │   ├── MouvementDAO.java
    │   │   └── UtilisateurDAO.java
    │   ├── controller/
    │   │   ├── LoginController.java
    │   │   ├── MainLayoutController.java
    │   │   ├── DashboardController.java
    │   │   ├── CategorieController.java
    │   │   ├── FournisseurController.java
    │   │   ├── ProduitController.java
    │   │   ├── MouvementController.java
    │   │   └── UtilisateurController.java
    │   └── utils/
    │       ├── DatabaseConnection.java
    │       ├── SessionManager.java
    │       └── ExportUtil.java
    └── resources/
        ├── fxml/
        │   ├── SplashScreen.fxml
        │   ├── Login.fxml
        │   ├── MainLayout.fxml
        │   ├── Dashboard.fxml
        │   ├── Categorie.fxml
        │   ├── Fournisseur.fxml
        │   ├── Produit.fxml
        │   ├── Mouvement.fxml
        │   └── Utilisateur.fxml
        ├── css/
        │   └── styles.css
        └── images/
            └── logo.png
```

2.3 Choix techniques justifiés

JavaFX 21 a été retenu pour la richesse de ses composants graphiques (`TableView`, `ComboBox`, `DatePicker`, `BarChart`) et sa compatibilité avec le modèle MVC via FXML et Scene Builder.

JDBC avec MySQL 8 garantit des performances adaptées à une application de bureau monoposte ou réseau local. L'utilisation systématique de `PreparedStatement` protège contre les injections SQL.

Le modèle Repository découple les contrôleurs de la logique d'accès aux données. Les interfaces permettent de substituer l'implémentation sans modifier le reste de l'application.

Apache POI 5.2.5 gère la génération de fichiers Excel `.xlsx` avec mise en forme conditionnelle (cellules rouges pour les produits en alerte).

iTextPDF 5.5.13.3 génère les exports PDF avec tableau formaté et en-têtes colorés (fonctionnalité bonus).

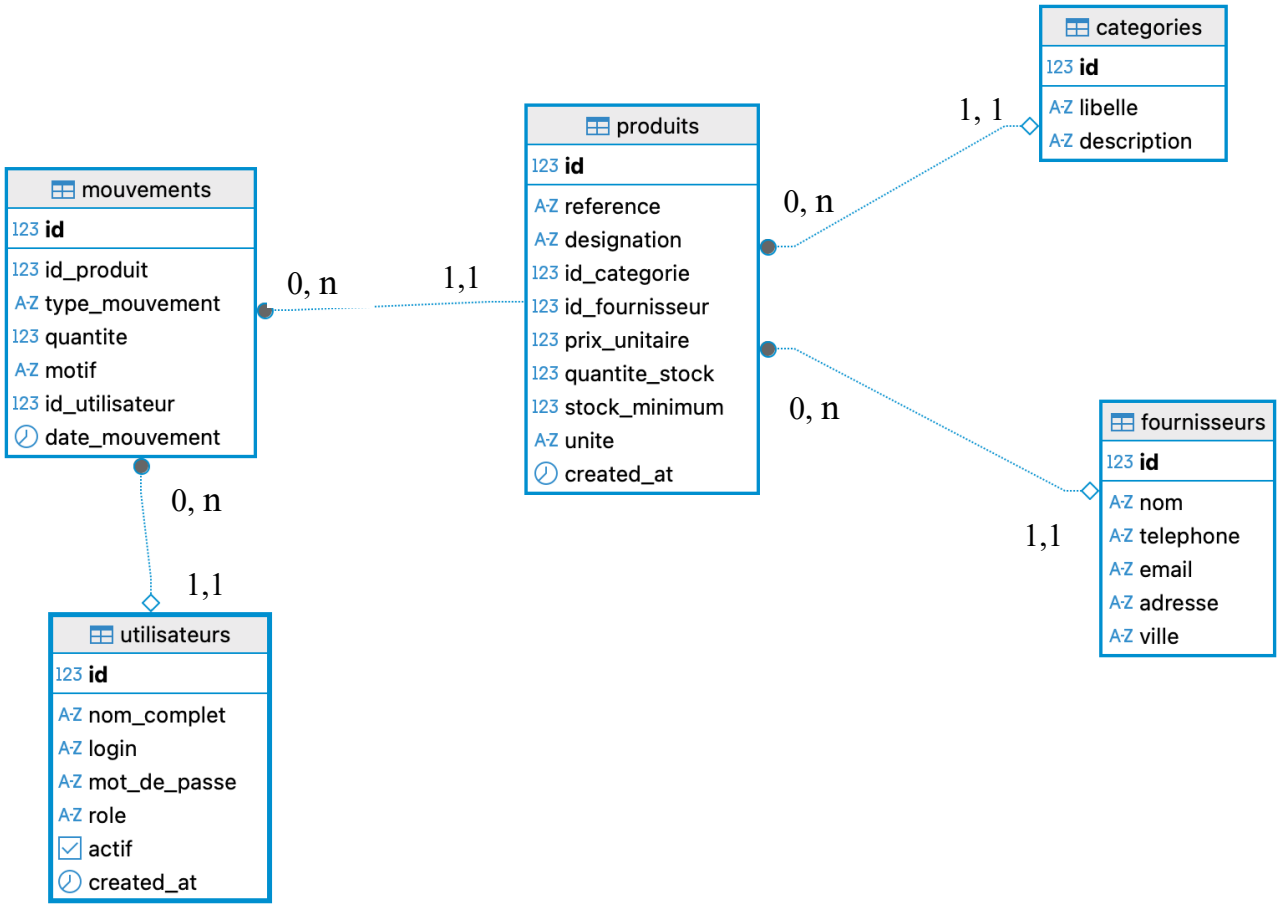
2.4 Sécurité

Les mots de passe ne sont jamais stockés en clair. La méthode `hashSHA256()` de `UtilisateurDAO` produit un hash hexadécimal de 64 caractères identique à la fonction `SHA2(mdp, 256)` de MySQL, ce qui assure la cohérence entre les données initiales et les authentifications ultérieures.

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
byte[] hash = md.digest(mdp.getBytes("UTF-8"));
StringBuilder sb = new StringBuilder();
for (byte b : hash) sb.append(String.format("%02x", b));
```

3. Schéma de la base de données

3.1 Diagramme des classes



3.2 Description des tables

Table catégories

Attribut	Type	Contrainte	Valeur par défaut	Nullable
id	Int(11)	PK	Auto	NON
libelle	VARCHAR(100)	UNIQUE	—	NON
description	VARCHAR(255)	—	—	OUI

Table fournisseurs

Attribut	Type	Contrainte	Valeur par défaut	Nullable
id	Int(11)	PK	Auto	NON
nom	VARCHAR(150)	—	—	NON
telephone	VARCHAR(20)	—	—	OUI
email	VARCHAR(100)	—	—	OUI
adresse	VARCHAR(255)	—	—	OUI
ville	VARCHAR(100)	—	—	OUI

Table produits

Attribut	Type	Contrainte	Valeur par défaut	Nullable
id	Int(11)	PK	Auto	NON
reference	VARCHAR(30)	UNIQUE	—	NON
designation	VARCHAR(200)	—	—	NON
id_categorie	Int (11)	FK → categories(id) SET NULL	—	OUI
id_fournisseur	Int(11)	FK → fournisseurs(id) SET NULL	—	OUI
prix_unitaire	DECIMAL(12,2)	—	0.00	NON
quantite_stock	Int(11)	—	0	NON
stock_minimum	Int(11)	—	5	NON
unite	VARCHAR(30)	—	'pièce'	OUI
created_at	TIMESTAMP	—	CURRENT_TIMESTAMP	NON

Table utilisateurs

Attribut	Type	Contrainte	Valeur par défaut	Nullable
id	Int(11)	PK	Auto	NON
nom_complet	VARCHAR(150)	—	—	NON
login	VARCHAR(50)	UNIQUE	—	NON
mot_de_passe	VARCHAR(64)	—	—	NON
role	ENUM	—	'GESTIONNAIRE'	OUI
actif	BOOLEAN	—	TRUE	OUI
created_at	TIMESTAMP	—	CURRENT_TIMESTAMP	NON

Table mouvements

Attribut	Type	Contrainte	Valeur par défaut	Nullable
ID	Int(11)	PK	Auto	NON
id_produit	Int(11)	FK → produits(id) CASCADE	—	NON
type_mouvement	ENUM	—	—	NON
quantite	Int(11)	—	—	NON
motif	VARCHAR(255)	—	—	OUI
id_utilisateur	Int(11)	FK → utilisateurs(id) SET NULL	—	OUI
date_mouvement	TIMESTAMP	—	CURRENT_TIMESTAMP	NON

3.3 Contraintes d'intégrité

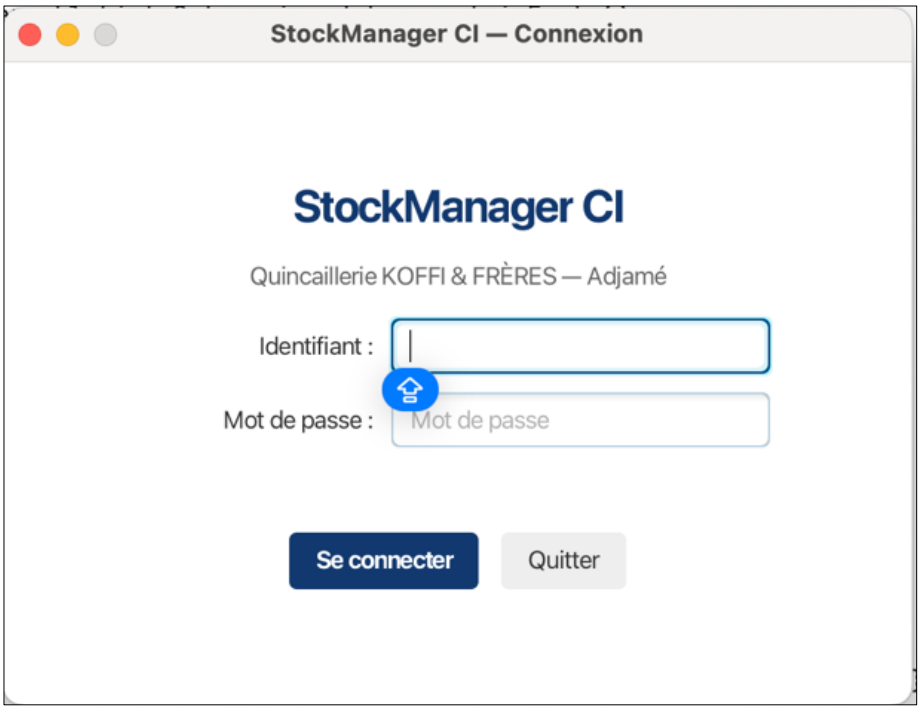
- `ON DELETE SET NULL` sur `id_categorie` et `id_fournisseur` : la suppression d'une catégorie ou d'un fournisseur ne supprime pas les produits liés ; leur référence passe simplement à `NULL`.
- `ON DELETE CASCADE` sur `id_produit` dans `mouvements` : la suppression d'un produit entraîne la suppression de son historique de mouvements.

4. Description des fonctionnalités

4.1 Authentification

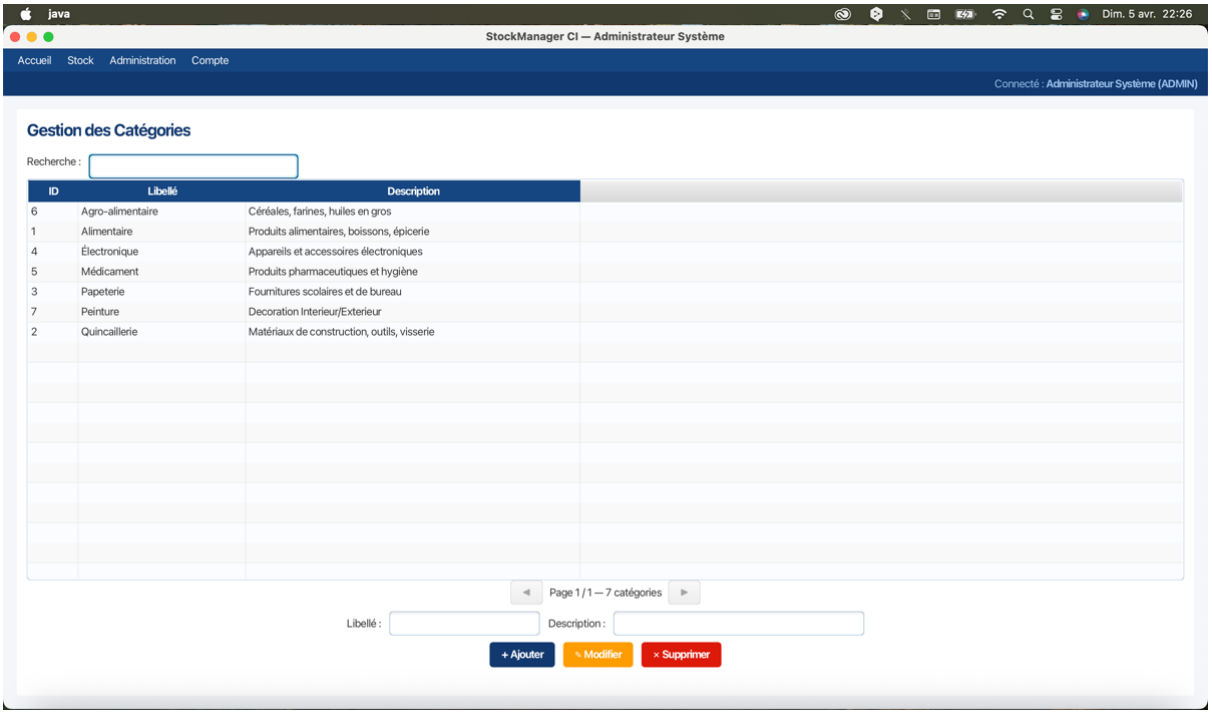
Au lancement, un **Splash Screen** non décoré (`StageStyle.UNDECORATED`) s'affiche pendant 3 secondes avec le logo de la quincaillerie, le nom de l'application et une barre de progression indéterminée. Une `PauseTransition` de JavaFX gère le délai avant l'ouverture de la fenêtre de connexion.

La fenêtre de connexion vérifie les identifiants en base de données. Après **3 tentatives échouées**, le bouton de connexion est désactivé pendant 30 secondes via une nouvelle `PauseTransition`. Le rôle de l'utilisateur connecté (`ADMIN` ou `GESTIONNAIRE`) est stocké dans la classe `SessionManager` et détermine les menus visibles dans la fenêtre principale.



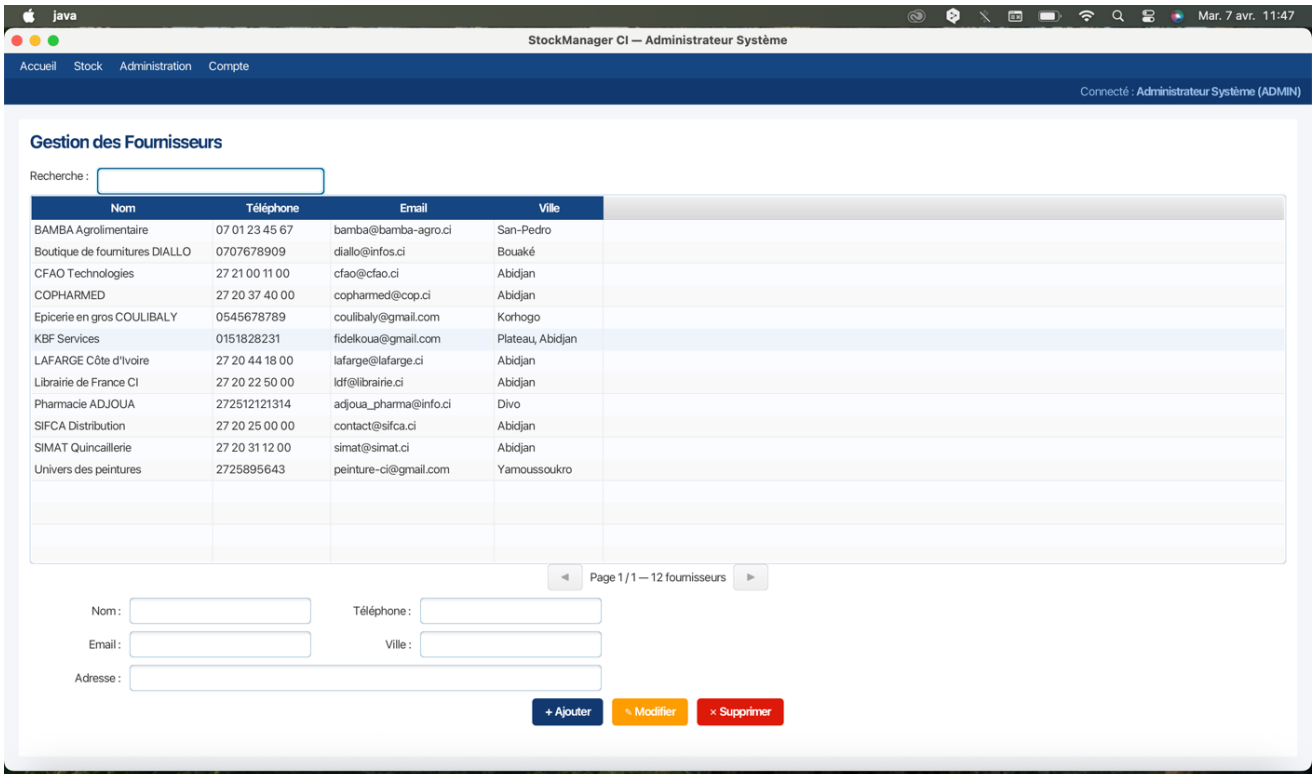
4.2 Module Catégories

Affiche la liste des catégories dans un `TableView` paginé (15 lignes par page). Le CRUD complet est opérationnel : ajout, modification et suppression. La suppression est bloquée si des produits sont rattachés à la catégorie concernée. Une recherche en temps réel filtre les résultats dès la saisie dans le champ de recherche.



4.3 Module Fournisseurs

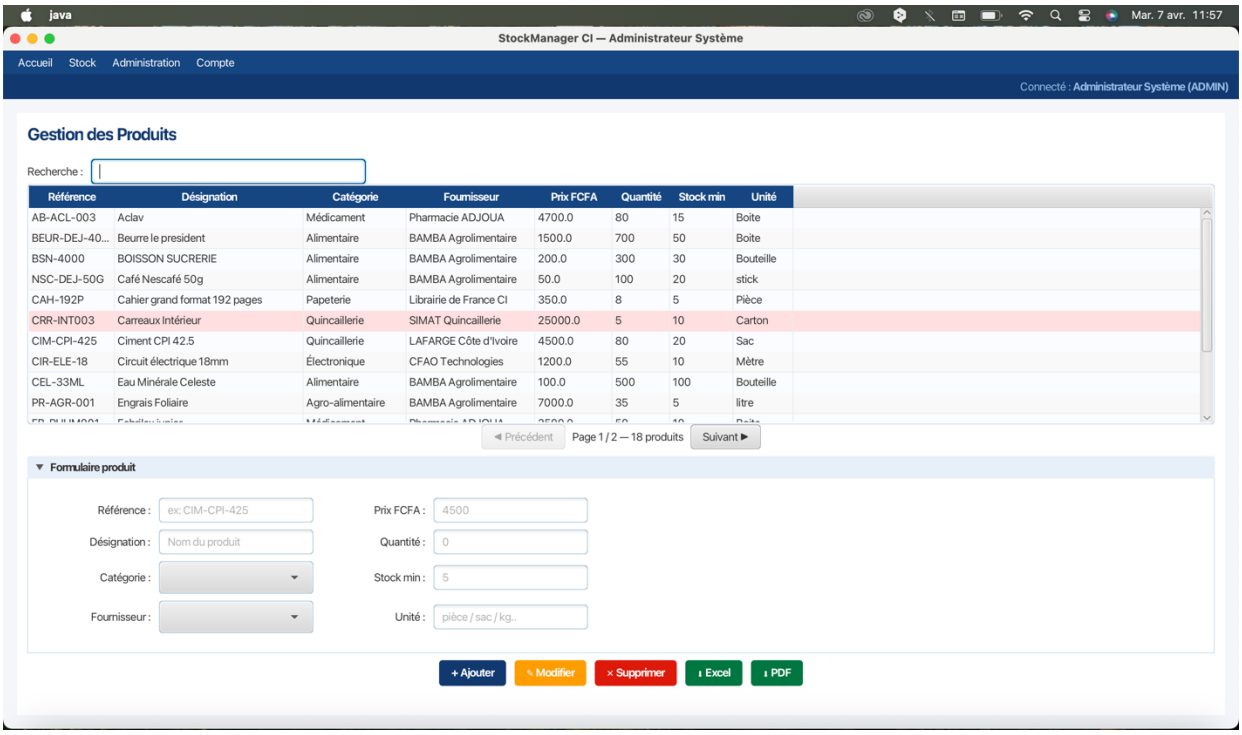
Fonctionnement identique au module Catégories. Les champs couvrent le nom, le téléphone, l'email, l'adresse et la ville. La recherche en temps réel opère sur le **nom** et le **numéro de téléphone**.



4.4 Module Produits

C'est le module le plus riche de l'application. Il propose un `TableView` paginé avec coloration des lignes en **rouge** dès que la quantité en stock est inférieure ou égale au stock minimum. Cette alerte est implémentée via un `RowFactory` personnalisé.

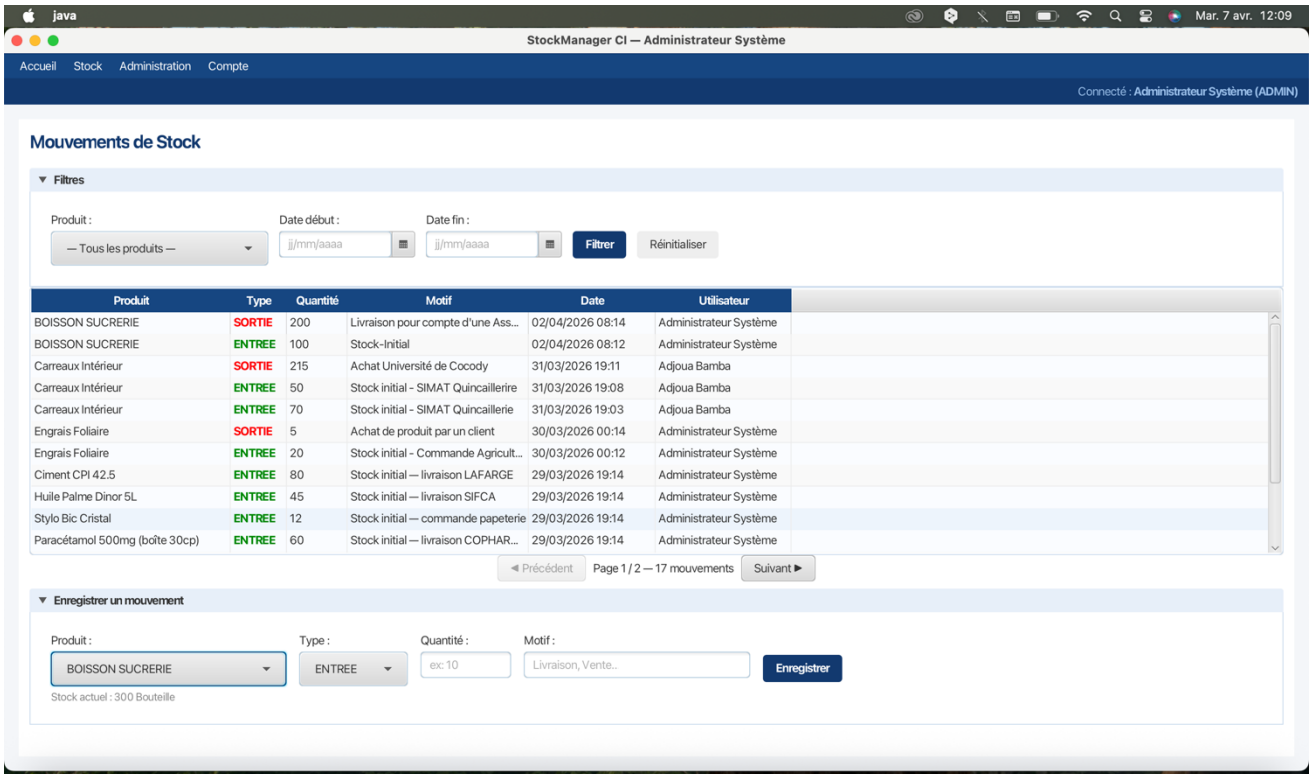
Le formulaire d'ajout/modification propose des `ComboBox` dynamiques pour la catégorie et le fournisseur, avec `setCellFactory` et `setButtonCell` pour afficher correctement les libellés. La recherche multicritères opère sur la **désignation**, la **catégorie** et le **fournisseur** simultanément.



4.5 Module Mouvements de Stock

Enregistre les entrées (approvisionnements) et sorties (ventes/consommations) de stock. Chaque enregistrement est une transaction au niveau de la base de données : **vérification du stock disponible**, insertion du mouvement, mise à jour de `quantite_stock` dans la table `produits`.

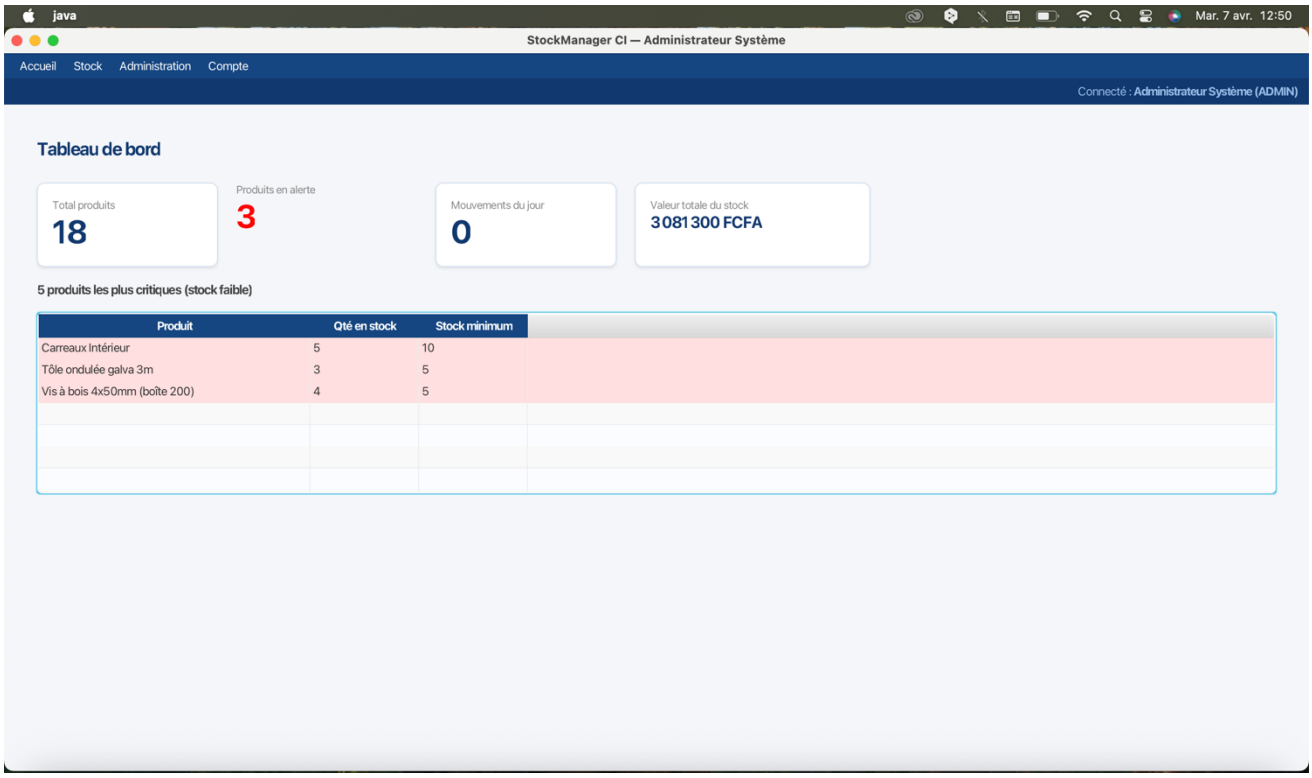
L'historique est filtrable par produit (via `ComboBox`) et par plage de dates (via deux `DatePicker`). Les mouvements ne peuvent pas être modifiés ni supprimés, garantissant la traçabilité complète.



4.6 Tableau de bord

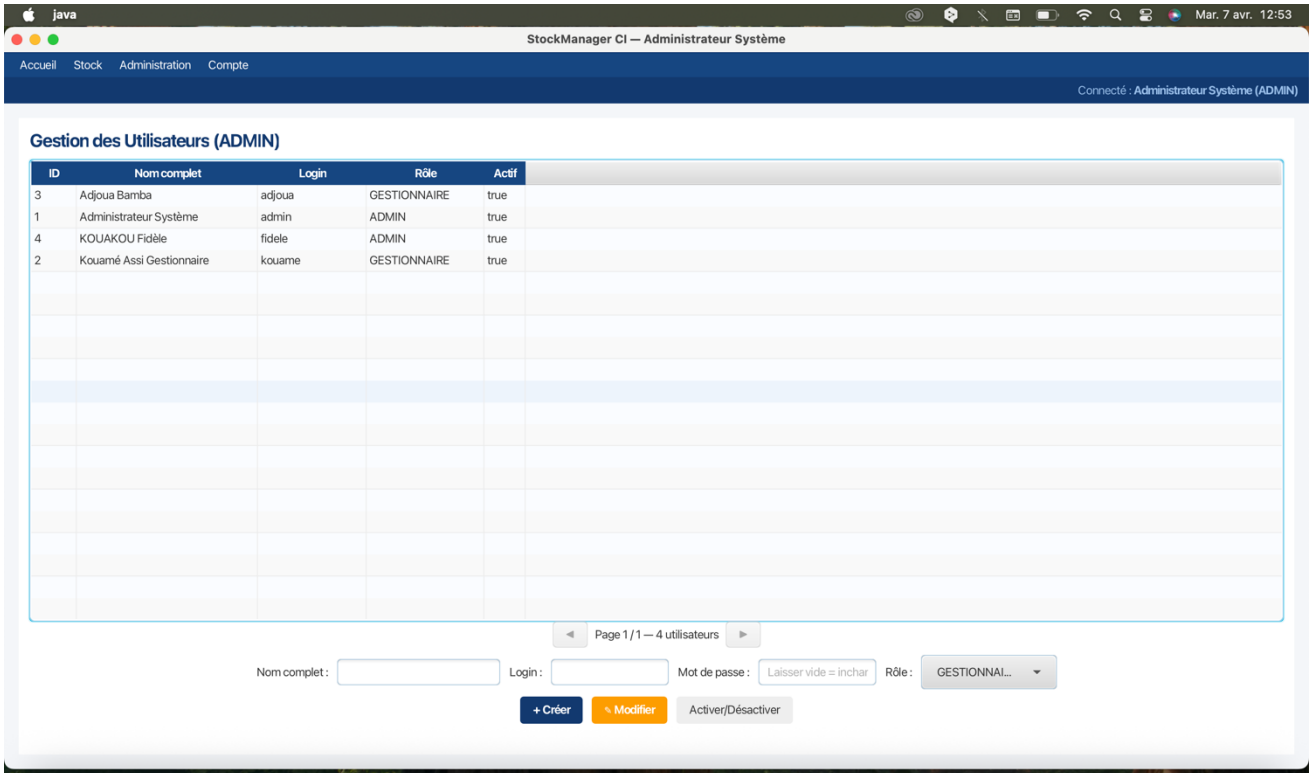
Affiché en premier après connexion, il présente quatre indicateurs clés sous forme de cartes:

- nombre total de produits,
- nombre de produits en alerte (affiché en rouge),
- nombre de mouvements du jour,
- valeur totale du stock en FCFA,
- Un tableau liste les cinq produits les plus critiques (ratio quantité/minimum le plus faible).



4.7 Module Utilisateurs (ADMIN uniquement)

Accessible uniquement aux utilisateurs ayant le rôle `ADMIN`. Permet de créer de nouveaux comptes, de modifier les informations et d'activer ou désactiver un compte. Un administrateur ne peut pas se désactiver lui-même.



4.8 Pagination

Tous les `TableView` de l'application utilisent une pagination (15 lignes par page). La navigation se fait via les boutons « Précédent » et « Suivant », et l'indicateur « `Page X / Y — N enregistrements` » est mis à jour à chaque changement de page ou de filtre.

5. Extensions implémentées

5.1 Splash Screen

Ce module affiche le logo de la quincaillerie, le nom de l'application et une barre de progression animée. La transition vers la fenêtre de login est gérée par une de 3 secondes.

5.2 Alertes visuelles stock minimum

Le produit applique un fond rouge clair dans le `TableView` à chaque ligne dont la `quantite_stock` est inférieure ou égale au `stock_minimum`. Ce contrôle est centralisé dans la méthode `isEnAlerte()` de la classe `Produit`, réutilisée dans le contrôleur, le tableau de bord et l'export Excel.

5.3 Export XLSX (Apache POI)

Le bouton « Excel » du module Produits ouvre une boite de dialogue permettant de choisir l'emplacement et le nom du fichier fichier excel (`.xlsx`) .

Le fichier généré contient un en-tête avec fond bleu foncé et texte blanc, et les lignes des produits en alerte sont colorées en rouge clair. La première ligne est figée pour faciliter la lecture.

5.4 Export PDF (bonus — iTextPDF)

Le bouton « PDF » génère un document A4 en paysage contenant un tableau de tous les produits avec le même code couleur que l'export Excel. La date de génération est incluse dans l'en-tête du document.

5.5 Filtre par date et produit dans les mouvements

La vue Mouvements propose une zone de filtres avec une `ComboBox` produit et deux `DatePicker` (date début / date fin). La requête SQL est construite dynamiquement : seules les clauses `WHERE` correspondant aux filtres actifs sont ajoutées, ce qui évite les requêtes incorrectes quand un filtre est vide.

5.6 Tests unitaires JUnit 5 (bonus)

Trois classes de tests couvrent les opérations principales des DAOs : `CategorieDAOTest` (save, findAll, update, search, delete), `ProduitDAOTest` (save, findAll, isEnAlerte, countEnAlerte, delete) et `UtilisateurDAOTest` (hashSHA256, authentifier, findByLogin).

6. Tableau de répartition des tâches

Fonctionnalité / Composant	Dosso Boiké	Kouakou Fidèle	Traoré Issiaka
Script SQL + données initiales	RP		
DatabaseConnection + SessionManager	RP	C	C
Classes Modèles (5 classes)	RP	C	C
Interfaces Repository (5 interfaces)	C	RP	C
Splash Screen + LoginController	C	RP	
MainLayout + Navigation	C	RP	C
Module Catégories (DAO+Controller+FXML)		RP	
Module Fournisseurs (DAO+Controller+FXML)		RP	C
Module Produits (DAO+Controller+FXML)		RP	C
Alertes visuelles stock minimum		RP	
Module Mouvements (DAO+Controller+FXML)		RP	
Filtre date + produit (Mouvements)		RP	C
Tableau de bord (Dashboard)	RP	C	C
Export XLSX (Apache POI)	C	C	RP
Export PDF — bonus (iTextPDF)	C	C	RP
Module Utilisateurs (admin)	C	RP	
Pagination (tous les TableView)	C	RP	
CSS + finition visuelle	RP	C	
Tests JUnit 5 — bonus	C	RP	
Rapport de conception	C	RP	C

Légende : RP = Responsable Principal | C = Contributeur | vide = non impliqué

7. Difficultés rencontrées et solutions

Difficulté 1 — `Location is not set` au démarrage

Problème : Au premier lancement de l'application, une erreur `IllegalStateException: Location is not set` empêchait le chargement du fichier `Login.fxml`. La méthode `getClass().getResource("/fxml/Login.fxml")` retournait `null`.

Cause : Le dossier `src/main/resources` n'était pas marqué comme « Resources Root » dans IntelliJ IDEA, ce qui empêchait Maven de copier les fichiers FXML dans le classpath lors de la compilation.

Solution : Clic droit sur `src/main/resources` → « Mark Directory as » → « Resources Root », suivi d'un « Rebuild Project ». La déclaration explicite du dossier `resources` dans `pom.xml` via la balise `<resources>` a également été ajoutée pour garantir la portabilité du projet.

Difficulté 2 — ComboBox affichant l'adresse mémoire des objets

Problème : Les `ComboBox` du formulaire produit (catégorie et fournisseur) affichaient `com.inphb.icgl.stocks.model.Categorie@52abda43` au lieu du libellé de la catégorie.

Cause : JavaFX n'utilise pas systématiquement la méthode `toString()` pour afficher les éléments d'une `ComboBox`. Il faut explicitement définir un `CellFactory` pour les cellules de la liste et un `ButtonCell` pour la valeur sélectionnée.

Solution : Ajout de `setCellFactory()` et `setButtonCell()` sur chaque `ComboBox`, avec des `ListCell` anonymes redéfinissant `updateItem()` pour afficher le libellé via `c.getLibelle()` ou `f.getNom()`.

Difficulté 3 — Logo introuvable dans le Splash Screen

Problème : L'erreur `Invalid resource: /images/logo.png not found on the classpath` apparaissait au démarrage après l'ajout du logo dans le Splash Screen.

Cause : La notation `@/images/logo.png` dans le FXML cherche le fichier à la racine du classpath. Or, `SplashScreen.fxml` se trouve dans `resources/fxml/`, et les ressources sont organisées de façon relative à ce fichier.

Solution : Utilisation d'un chemin relatif `@../images/logo.png` qui remonte d'un niveau (`..`) depuis `fxml/` pour atteindre `resources/`, puis descend dans `images/`.

8. Bilan individuel

Dosso Boiké

Ma contribution principale a porté sur la **couche de données** du projet. J'ai rédigé le script SQL complet avec les données initiales contextualisées (produits, fournisseurs et utilisateurs ivoiriens), mis en place la classe `DatabaseConnection` (Singleton JDBC avec chargement explicite du driver) et le `SessionManager`. J'ai également conçu les cinq classes modèles JavaFX avec leurs `Property`, leurs trois constructeurs et leurs getters/setters. La prise en charge du tableau de bord et de la charte graphique CSS m'a permis d'allier rigueur technique et souci de l'ergonomie. La difficulté principale rencontrée a été la cohérence du hash SHA-256 entre Java (`MessageDigest`) et MySQL (`SHA2(mdp, 256)`), résolue en vérifiant l'encodage UTF-8 et le format hexadécimal de sortie.

Kouakou Fidèle

J'ai été le responsable principal de la grande majorité des modules fonctionnels de l'application. J'ai conçu les cinq interfaces Repository qui structurent l'accès aux données, puis implémenté tous les DAO correspondants, recherche dynamique et gestion des contraintes d'intégrité. J'ai également développé le Splash Screen, le système d'authentification avec blocage après 3 tentatives, la navigation principale `MainLayout`, les modules Catégories, Fournisseurs, Produits (avec alertes visuelles via `RowFactory`), Mouvements, Utilisateurs et la pagination de tous les `TableView`. La difficulté principale a été la gestion de la transaction JDBC dans `MouvementDAO.save()` : vérification du stock, insertion du mouvement et mise à jour de `quantite_stock` dans un seul bloc `setAutoCommit(false)` / `commit()` / `rollback()`. J'ai aussi rédigé l'essentiel du rapport de conception.

Traoré Issiaka

Ma contribution a porté sur les **exports de données** et le support des modules principaux. J'ai développé les exports Excel (Apache POI) et PDF (iTextPDF) avec mise en forme conditionnelle (cellules rouges pour les produits en alerte, en-têtes colorés, première ligne figée). J'ai également contribué au filtre combiné date/produit dans le module Mouvements, à la navigation principale, aux modules Fournisseurs et Produits en tant que contributeur. La difficulté principale rencontrée a été l'affichage incorrect des `ComboBox` qui montrait l'adresse mémoire des objets au lieu du libellé, résolu par l'ajout de `setCellFactory()` et `setButtonCell()` avec des `ListCell` personnalisés.

9. Conclusion et perspectives

9.1 Bilan

L'application **StockManager CI** répond intégralement au cahier des charges. Les neuf modules obligatoires sont fonctionnels, l'architecture MVC + Repository est respectée, la sécurité des mots de passe est assurée par SHA-256, et toutes les extensions sont implémentées. L'application a été testée avec des données réalistes représentant le contexte ivoirien (produits, fournisseurs et utilisateurs contextualisés).

9.2 Perspectives d'amélioration

Plusieurs axes d'amélioration pourraient enrichir l'application dans une version ultérieure :

Graphiques sur le tableau de bord : L'ajout d'un `BarChart` ou `PieChart` JavaFX représentant la répartition des stocks par catégorie ou l'évolution des mouvements sur les 30 derniers jours apporterait une valeur analytique supplémentaire.

Multi-utilisateurs réseau : Actuellement monoposte, l'application pourrait être adaptée pour un usage multi-utilisateurs en réseau local en remplaçant la connexion Singleton par un pool de connexions (HikariCP).

Import depuis Excel : L'ajout d'une fonctionnalité d'import de produits depuis un fichier `.xlsx` faciliterait la migration depuis une gestion existante sous tableur.

Notifications automatiques : Un système d'alerte par email ou par notification système pourrait être déclenché automatiquement lorsqu'un produit passe en dessous de son stock minimum.

Sauvegarde automatique : L'intégration d'une tâche planifiée pour effectuer des sauvegardes régulières de la base de données garantirait la pérennité des données.

10. Annexes

Annexe A — Dépendances Maven

```
<!-- JavaFX 21 -->
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.6</version>
</dependency>
<!-- Driver JDBC MySQL 8 -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
    <scope>compile</scope>
</dependency>

<!-- Apache POI — export XLSX (OBLIGATOIRE) -->
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.5</version>
</dependency>

<!-- iTextPDF — export PDF (BONUS) -->
<dependency>
    <groupId>com.itextpdf</groupId>
    <artifactId>itextpdf</artifactId>
    <version>5.5.13.3</version>
</dependency>

<!-- JUnit 5 — tests unitaires (BONUS) -->
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
</dependency>
```

Annexe B — Identifiants de test

Login	Mot de passe	Rôle
admin	admin123	ADMIN
kouame	kouame2026	GESTIONNAIRE
adjoua	adjoua2026	GESTIONNAIRE
Fidele	fidele123	ADMIN

Annexe C — Commandes de déploiement

```
# Compiler et générer le JAR
mvn clean package

# Lancer l'application
mvn javafx:run

# Générer un installeur .dmg (recommandé — image disque glisser-
déposer)
jpackage --input target/ \

  --name StockManagerCI \

  --main-jar stockmanager-ci-1.0.jar \

  --main-class com.inphb.icgl.stocks.MainApp \

  --type dmg \

  --icon src/main/resources/images/logo.icns \

  --app-version 1.0
```

Annexe D — Extrait de code

```
@Override
public boolean save(Mouvement m) {
    Connection cn = null;
    try {
        cn = DatabaseConnection.getConnection();
        cn.setAutoCommit(false);

        // 1. Vérifier le stock disponible pour les SORTIES
        if ("SORTIE".equals(m.getTypeMouvement())) {
            PreparedStatement chk = cn.prepareStatement(
                    "SELECT quantite_stock FROM produits WHERE id=?");
            chk.setInt(1, m.getIdProduit());
            ResultSet rs = chk.executeQuery();
            if (rs.next() && rs.getInt(1) < m.getQuantite()) {
                cn.rollback();
                return false; // Stock insuffisant
            }
            chk.close();
        }

        // 2. Insérer le mouvement
        PreparedStatement ps = cn.prepareStatement(
                "INSERT INTO mouvements " +
                        "(id_produit, type_mouvement, quantite, motif,
id_utilisateur) " +
                        "VALUES (?,?,?,?,?)");
```

```
        ps.setInt(1,    m.getIdProduit());
        ps.setString(2, m.getTypeMouvement());
        ps.setInt(3,    m.getQuantite());
        ps.setString(4, m.getMotif());
        ps.setInt(5,    m.getIdUtilisateur());
        ps.executeUpdate();
        ps.close();

        // 3. Mettre à jour la quantité du produit
        String op = "ENTREE".equals(m.getTypeMouvement()) ? "+" : "-";
        PreparedStatement upd = cn.prepareStatement(
                "UPDATE produits SET quantite_stock = quantite_stock " + op + " ?
WHERE id=?");
        upd.setInt(1, m.getQuantite());
        upd.setInt(2, m.getIdProduit());
        upd.executeUpdate();
        upd.close();

        cn.commit();
        return true;

    } catch (SQLException e) {
        System.err.println("[MouvementDAO.save] " + e.getMessage());
        if (cn != null) try { cn.rollback(); } catch (SQLException ignored) {}
        return false;
    } finally {
        if (cn != null) try { cn.setAutoCommit(true); } catch (SQLException
ignored) {}
    }
}
```